Language Theory & Automata — Cheat Sheet

Dr-Ing Ines Baccouche · Pristini · 2nd year Applied AI · Chapters 1–3 (RegEx · Finite Automata, Glushkov & Minimisation · DFA / NFA)

✍️ How she writes things in class — copy this format on the exam

- **Automaton** = 5-tuple `A = (Q, V_T, δ, q0, F)` — always write all 5 parts.
- **States**: `q0, q1…` or plain numbers `0,1,2…` (Glushkov). Initial = incoming arrow (in tables `→q0`). Final = **double circle** (in tables `*q2` or a **box** around it).
- **Transition table**: rows = states, columns = symbols, `δ` in the corner. Empty cell = `-` or `ϕ`. Several targets = a set `{q1, q2}`.
- **NFA→DFA**: one line per computation: `δ'([q1], 0) = [q1, q2] → new state`, then **rename** the new set (`{q1,q2} = q3`) and redraw.
- **Glushkov**: number every letter with an index: `a1* b2 (a3|c4)* b5`. Then a **"successors" table** whose first row is `ϕ` (= start letters). Final letters are **boxed**.
- **Minimisation**: one line per step, groups in brackets: `Step0: (6,7,8) (1,2,3,4,5)` `Step1: (6) (7,8) (1,2,3,4,5)` … then "**fusion** (same behaviour)".
- **Identical rows** in a table → she **boxes them** → they are merged into one state.
- **Merged state** is named with all its members: `"1,2"`, `"7,8"`, `"0,1,3,6"`.

Chapter 1 — Language theory & Regular Expressions

1. Vocabulary

- **Symbol**: `a` · **Alphabet** Σ: finite set of symbols `{a, b}`
- **String / word**: finite sequence of symbols `aabb`
- **Language** L ⊆ Σ*: a set of strings `{ε, ab, aabb, aaabbb, …}` (finite or infinite)
- **ε** = empty string, `|ε| = 0` ; `|01101| = 5`
- **Σ^k** = strings of length k. $\Sigma^0 = \{\varepsilon\}$; Σ={0,1} ⇒ $\Sigma^2 = \{00,01,10,11\}$
- **Σ*** = $\Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$ (Kleene star, all strings)
- **Σ⁺** = $\Sigma^1 \cup \Sigma^2 \cup \dots$ ⇒ $\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$
- **Concatenation**: x=01101, y=110 → xy = 01101110, yx = 11001101. `εw = wε = w`

⚠ Classic traps

- **ø ≠ {ε}**: ø has *no* string, {ε} has *one* (the empty one).
- **Σ ≠ Σ^1**: Σ contains *symbols*, Σ^1 contains *strings* of length 1.
- xy ≠ yx in general (concatenation is not commutative).
- L* **always** contains ε; L^+ contains ε only if ε ∈ L.
- {0,11}* = 0s and 1s where the 1s come **in pairs**: 011, 11110, ε ✔ — 01011, 101 ✘

Exercise (slide 14) — A = {a,b}, u = aa, v = bab

- uv = **aabab** · $(uv)^2$ = **aababaabab** · u^3v = **aaaaaabab**
- Words of length 2: **{aa, ab, ba, bb}**
- E1 = {u.v / u,v ∈ A^+} = **words with |w| ≥ 2**; E2 = **A^+**; E3 = **A***

2. Operations on languages & RegEx symbols

Operation	Symbol	Meaning	Example
Union (alternation)	`|`	in L or in M (or both)	L={001,10,111}, M={ε,001} ⇒ L∪M = {ε,001,10,111}
Concatenation	`.` / juxtaposition	a word of L followed by a word of M	L.M = {001,10,111,001001,10001,111001}
Kleene star (closure)	`*`	0 or more repetitions (ε included)	`a*` → ε, a, aa, aaa…
Kleene plus (positive closure)	`+`	1 or more repetitions	`b+` → b, bb…; `\d+` → 1, 123…
Optional	`?`	0 or 1 time	`ab?` → a, ab
Exact / min count	`{n}` `{n,}`	exactly n / at least n times	`(a|b|c){3}`, `[a-z]{2,}`
Intersection · Difference · Complement	`n` · `\` · `¬`	common · in first not second · not in L	{a,b}∩{b} = {b}; {a,b}\{b} = {a}
Grouping	`( )`	order of evaluation	`(ab)*` → ε, ab, abab…

Priority: * (and + ?) > . concatenation > | union. So `ab*|c` = `(a(b*)) | c`.

Regular expression (basis): ε and ø are RegEx (L(ε)={ε}, L(ø)=ø); every symbol a is a RegEx (L(a)={a}); if E, F are RegEx then E|F, EF, E* and (E) are RegEx. A language is **regular** ⇔ it is denoted by a RegEx.

3. RegEx notation (slide 33)

Token	Matches	Set
`\d`	any digit	`[0-9]`
`\s`	whitespace	`[ \t\n\r\f]`
`\w`	letter, digit, _	`[A-Za-z0-9_]`
`\D \S \W`	the inverse	`[^0-9]` ...
`.`	any one character	
`[A-Z]`	one capital letter	

UNIX example: `[A-Z][a-z]*[ ][A-Z][A-Z]` = capitalised word + space + 2 capitals (a city and a state: "Paris TX").

Uses: lexical analysers (compilers), search, text recognition, form validation.

4. Classic RegEx patterns over {a,b,c} (Exercise 2)

All strings	`(a|b|c)*` = `(a*b*c*)*`
Three-letter strings	`(a|b|c){3}`
Without 'b'	`(a|c)*`
At most one 'a'	`(b|c)* | (b|c)*a(b|c)*`
At least one 'a'	`(a|b|c)*a(a|b|c)*`
Each 'a' followed by 'b'	`(b|c|ab)*`
Even length	`((a|b|c)(a|b|c))*`
Prefix 'ab'	`ab(a|b|c)*`
Suffix 'ab'	`(a|b|c)*ab`
Exactly 3 'a'	`(b|c)*a(b|c)*a(b|c)*a(b|c)*`
At least 3 'a'	`(a|b|c)*a(a|b|c)*a(a|b|c)*a(a|b`

5. Solved exercises (slides 27–36)

R = a(a|b)ba = {aaba, abba}, S = (ab)*|(ba)*

1. In R not S: **aaba**
2. In S not R: **ab** (or ε, abab)
3. In both: **none!** R's words (length 4) are aaba, abba; S's words of length 4 are abab, baba ⇒ L(R)∩L(S) = ø
4. In neither: **b**, aa, bbb...

Give the elements:

- `ab*|c` = **{c, a, ab, abb, abbb, ...}**
- `(a+ b)` = **{ab, aab, aaab, ...}** (+ = one or more)
- `b*ab*ab*` = **words of {a,b} with exactly two a**: aa, aba, baab...
- `(a|b*)*(c|d)*` = **any word of a,b followed by any word of c,d**: ε, ab, ba, cd, abdc...

6. True or false? (slide 29)

- `(a*|b*) = (a|b)*` → **False**: *ab* ∈ (a|b)* but not in a*|b*
- `(a|b)*b = (a|b)*` → **False**: ε and *a* are not in (a|b)*b
- `(a*b)* = (a|b)*b` → **False**: ε ∈ (a*b)* but ε ∉ (a|b)*b. (True only if we add ε: (a*b)* = ε | (a|b)*b)

To prove "False" → give ONE word that is in one side and not the other (counter-example).

Exercises 3–5

- L1 = begins with a: **`a(a|b)*`** · L2 = even length: **`((a|b)(a|b))*`**
- 3-digit numbers: `\d{3}` · divisible by 5: `[0-9]*(0|5)` · start 4 end 7: `4[0-9]*7` (slide: `4.*7`)
- Email: `[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}`

Chapter 2 — Finite Automata principles · Glushkov · minimisation

1. Definition

A **finite automaton** (finite state machine) reads a string and answers **yes** if S ∈ L(G), **no** otherwise.

$$A = (Q, V_T, \delta, q0, F)$$

- V_T: finite alphabet · **Q**: finite set of states
- **q0**: initial state · **F**: set of final (acceptor) states
- **δ**: transition function $\delta : Q \times V_T \to Q$

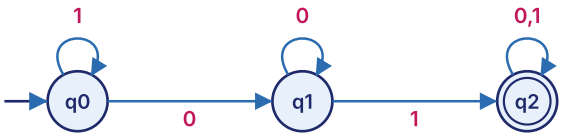
Accepted word: w = a1a2...an is accepted if there is a **path** that starts in the initial state, follows the labels a1, a2, ..., an and **ends in a final state**.

Used in: hardware design, lexical analysis (compilers), text search, robots/planning, program verification, text compression.

2. Example — strings over {0,1} containing "01"

She reasons with **scenarios → one state each**:

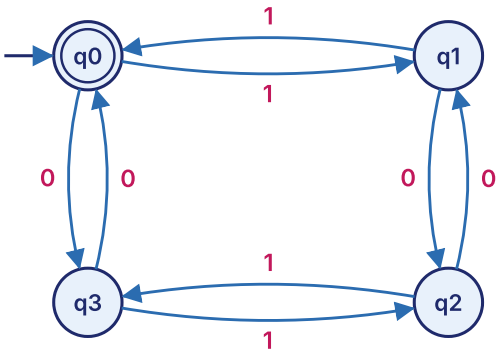
- q0: the first 0 has not appeared yet
- q1: a 0 appeared, waiting for a 1
- q2: "01" was met (final, stays forever)



3. Example — even number of 0 AND of 1

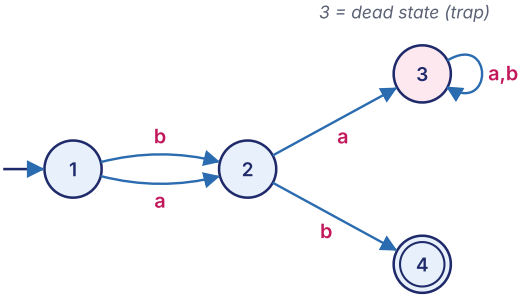
`M = ({q0,q1,q2,q3}, {0,1}, δ, q0, {q0})`

δ	0	1
→*q0	q3	q1
q1	q2	q0
q2	q1	q3
q3	q0	q2



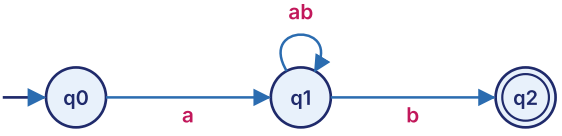
q0 = (even 0, even 1) · q1 = (even 0, odd 1) · q2 = (odd, odd) · q3 = (odd 0, even 1)

4. Reading a diagram (slide 49) & table (slide 47)



Σ={a,b}, Q={1,2,3,4}, F={4}. Language = **(a|b)b** = {ab, bb}. State 3 is a **trap**: once there, never accepted.

L = a(ab)*b, Q={q0,q1,q2}, V_T={a,b,ab}: δ(q0,a)=q1, δ(q1,b)=q2, δ(q1,ab)=q1



Glushkov construction RegEx → automaton

Method — 3 steps

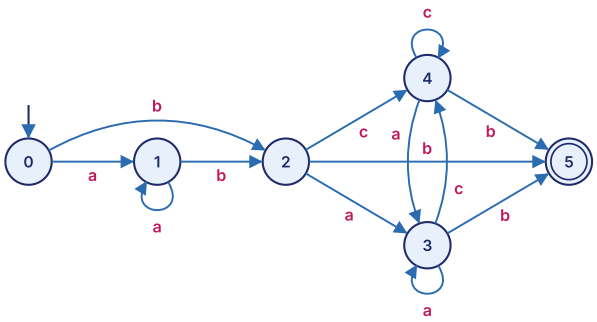
1. **Number** every symbol of the RegEx from 1 (each symbol appears once) → RE2.
   `a*b(a|c)*b → a1* b2 (a3|c4)* b5`
   States: q0 = 0, Q = {0,1,…,n} (n = number of letters).
2. **Build A'**:
   a. **Start letters** (can begin a word) → arcs `0 → i` labelled a_i
   b. **End letters** (can finish a word) → F = their numbers. **If ε ∈ L(RE) add 0 to F!**
   c. **Factors of length 2** a_ia_j (a_j can come right after a_i) → arc `i → j` labelled a_j
3. **Erase the indices** (a3 → a) and draw the automaton A.

Rule: an arc always goes INTO the state of the letter it reads: reading a_j ⇒ go to state j.

Example 1 — a*b(a|c)*b → a1* b2 (a3|c4)* b5

- Start letters = {a1, b2} · End letters = {b5} ⇒ F = {5}
- Factors: a1a1 a1b2 b2a3 b2c4 b2b5 a3a3 a3c4 c4a3 c4c4 a3b5 c4b5

	successors
ϕ	a1, b2
a1	a1, b2
b2	a3, c4, b5
a3	a3, c4, b5
c4	a3, c4, b5
b5	ϕ

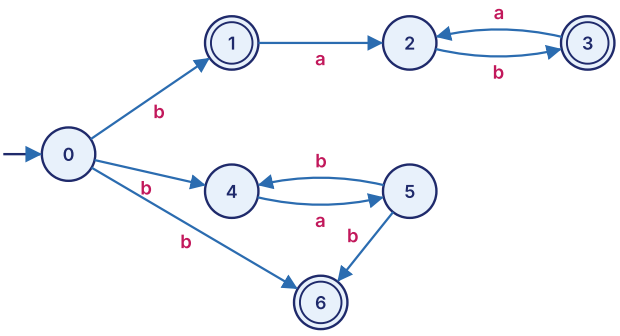


Example — b(ab)* | (ba)*b → b1(a2b3)* | (b4a5)*b6

	successors
ϕ	b1, b4, b6
b1	a2
a2	b3
b3	a2
b4	a5
a5	b4, b6
b6	ϕ

Boxed = end letters ⇒ **F = {1, 3, 6}**.
b1 is an end letter because (a2b3)* can be empty.

⚠ From 0, reading **b** goes to 1, 4 *and* 6 ⇒ Glushkov usually gives an **NFA** → determinise it (Ch3).



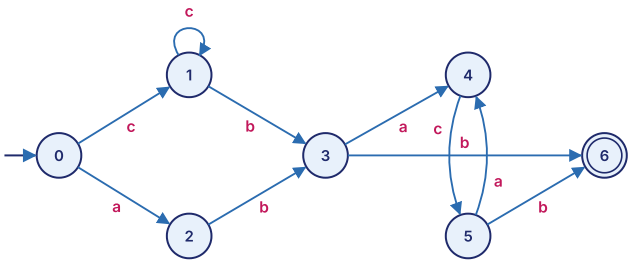
Example 2 (slide 57, left blank) — (c+|a) b (ac)* b — solved

`(c1+ | a2) b3 (a4 c5)* b6` (c+ = c c*, so c1 can follow c1)

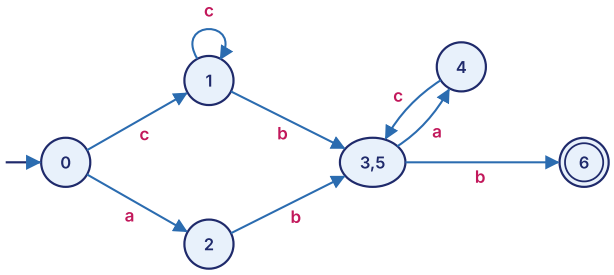
	successors
ϕ	c1, a2
c1	c1, b3
a2	b3
b3	a4, b6
a4	c5
c5	a4, b6
b6	ϕ

q0 = 0, Q = {0,...,6}, **F = {6}** (ε ∉ L).

Rows **b3** and **c5** have the same successors and both are non-final ⇒ states 3 and 5 can be **merged** (minimisation).



After minimisation (3 ≡ 5):



Minimisation of automata same language, fewest states

Method

Why: less memory/computation, easier to read; used in compilers, pattern matching, model checking.

1. **Remove unreachable states** (mark everything reachable from the start; delete the rest).
2. **Step0**: split into 2 groups → **(final states) (non-final states)**.
3. For every state of a group, look **which group** each letter sends it to. States of a group that go to **different groups** are **distinguishable** → split the group.
4. Repeat (Step1, Step2, ...) until nothing splits any more.
5. Each final group = **one state** ("fusion"). Initial = group containing the old initial; final = groups of old finals.

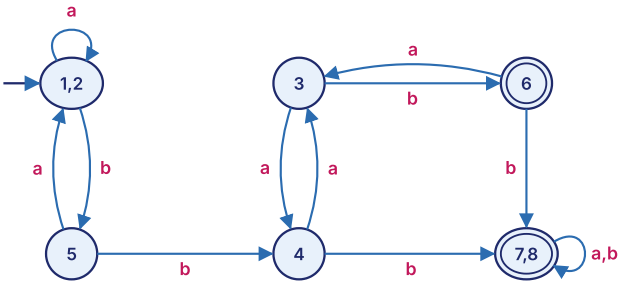
Two states are distinguishable if one word leads one of them to a final state and the other to a non-final one.

Example (slide 60) — F = {6, 7, 8}

	a	b
→1	1	5
2	2	5
3	4	6
4	3	8
5	2	4
*6	3	7
*7	8	7
*8	7	7

```
Step0 : (6,7,8)
(1,2,3,4,5)
Step1 : (6) (7,8)
(1,2,3,4,5)
Step2 : (6) (7,8) (3)
(4) (1,2,5)
Step3 : (6) (7,8) (3)
(4) (1,2) (5)
→ stable ⇒ 6 states
```

- Step1: 6 –a→ 3 (non-final) but 7,8 –a→ final ⇒ 6 alone.
- Step2: 3 –b→ 6 ∈ (6), 4 –b→ 8 ∈ (7,8), 1,2,5 stay inside ⇒ 3 and 4 alone.
- Step3: 5 –b→ 4 (now alone) but 1,2 –b→ 5 ⇒ 5 alone. 1 and 2 behave the same ⇒ merge.



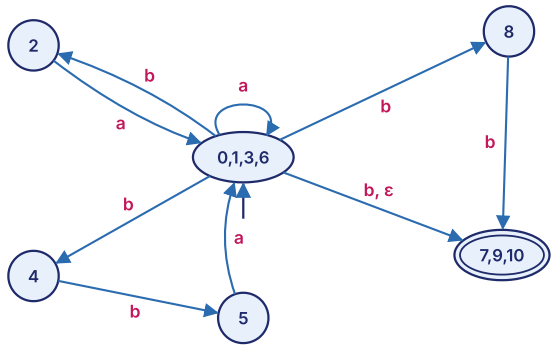
Class exercise (board photo, slide 61) — minimise (a | ba | bba)* (b | bb | ε)

① **Number:** `(a1 | b2a3 | b4b5a6)* (b7 | b8b9 | ε10)`

② **Successors table** (as on the board):

ϕ	a1 b2 b4 b7 b8 ε10
a1	a1 b2 b4 b7 b8 ε10
b2	a3
a3	a1 b2 b4 b7 b8 ε10
b4	b5
b5	a6
a6	a1 b2 b4 b7 b8 ε10
b7	ϕ
b8	b9
b9	ϕ
ε10	ϕ

Final letters ⇒ **F = {7, 9, 10}**



③ **Transition table** — boxed rows are identical:

	a	b	ε
→0	1	2,4,7,8	10
1	1	2,4,7,8	10
2	3	-	-
3	1	2,4,7,8	10
4	-	5	-
5	6	-	-
6	1	2,4,7,8	10
***7**	-	-	-
8	-	9	-
***9**	-	-	-
***10**	-	-	-

④ **Steps (her writing):**

```
1) {7,9,10} {0,1,2,3,4,5,6,8}
2) {7,9,10} {8} {2,4,5} {0} {1} {3} {6}
3) {7,9,10} {8} {2} {4} {5} {0} {1} {3} {6}
4) {7,9,10} {0,1,3,6} {8} {2} {4} {5}  ← fusion (same behaviour)
```

Why the fusions? 7, 9, 10 have no exit at all ⇒ same behaviour. 0, 1, 3, 6 have the **same row** (a→1, b→2,4,7,8, ε→10) ⇒ one state "0,1,3,6".

> **Note:** with strict step-by-step refinement 0,1,3,6 are never separated, and **2 and 5** also behave the same (each only reads *a* and goes back into {0,1,3,6}) — so they could be merged into "2,5" too. Write her version, and mention this if you have time.

Language check: "0,1,3,6" reads *a*, *ba* (via 2) or *bba* (via 4,5) and comes back; the end reads b, bb (via 8) or nothing (ε).

Chapter 3 — Deterministic & non-deterministic automata

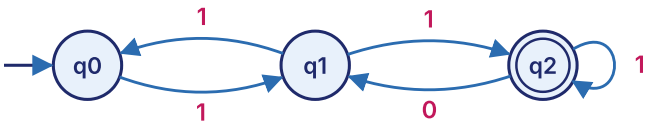
1. Definitions

- **Completely defined**: for each state q and each symbol s ∈ Σ there is **at least one** transition labelled s leaving q.
- **Unambiguous**: for each state q and symbol s there is **at most one** transition labelled s leaving q.
- **DFA** (deterministic) = completely defined **AND** unambiguous ⇒ **exactly one** arrow per (state, symbol). **No ε-arrows.**
- **NFA** (non-deterministic) = presents **ambiguity**: several arrows with the same symbol from one state, or ε-arrows. "We cannot deduce the next state."

How to complete a DFA: add a **dead state** (trap) that receives every missing transition and loops on all symbols.

Every NFA has an equivalent DFA (same language) → subset construction below.

2. Activity (slide 66) — solved



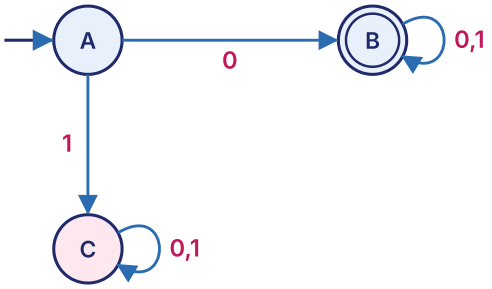
δ	0	1
→q0	-	q1
q1	-	{q0, q2}
*q2	q1	q2

- q1 –1→ q0 **or** q2 ⇒ ambiguity ⇒ **NFA**; q0 and q1 have no 0-arrow ⇒ not completely defined.
- **101**: q0 –1→ q1 –0→ ? no transition ⇒ **rejected (not modeled)**
- **1111**: q0→q1→q2→q2→q2 ⇒ **accepted**
- **1101**: q0→q1→q2 –0→ q1 –1→ q2 ⇒ **accepted**

3. Build a DFA from a sentence — her recipe

1. List the strings of L: L = {0, 00, 01, 000, ...}
2. Draw the **shortest accepted word** as a path.
3. Add the other arrows so that **every state has one arrow per symbol**.
4. Words that can never be accepted → **dead state**.

Starts with 0 over {0,1}:

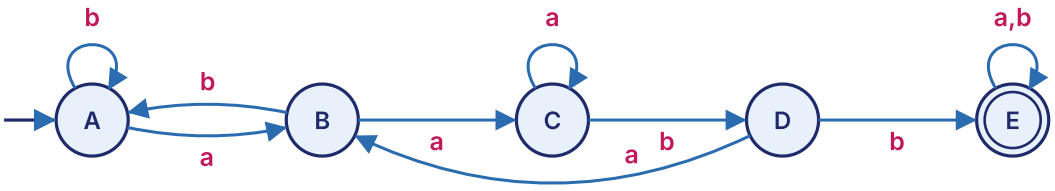


C = dead state (catches every 'wrong' word)

Length exactly 2: L = {00, 01, 10, 11}

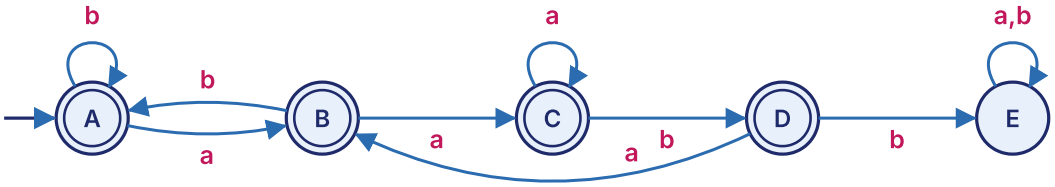


4. Contains "aabb" — and the complement trick



State = how much of "aabb" is matched (A: nothing, B: a, C: aa, D: aab, E: found). D –a→ B because "aaba" ends with "a".

Does NOT contain aabb → **flip the states**: 1. final → non-final, 2. non-final → final.



⚠ Flipping only works on a **complete DFA** (add the dead state first, never on an NFA).

NFA → DFA subset construction

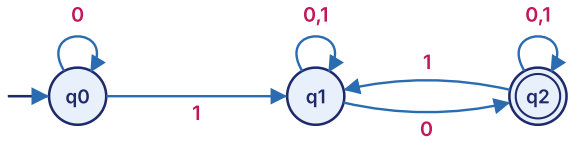
Method (M = (Q,Σ,δ,q0,F) → M' with L(M) = L(M'))

1. Q' = ϕ.
2. Add **[q0]** to Q' and compute its transitions.
3. For every state of Q' and every symbol, compute the **set** of reachable states: `δ'([q1,q2], 0) = δ(q1,0) ∪ δ(q2,0)`. If the set is **new** → add it to Q' ("new state").
4. Stop when no new state appears.
5. **Final states** of the DFA = every set that **contains** a final state of the NFA.

States never reached from [q0] (e.g. [q2] below) are **unreachable** → drop them. ϕ = dead state.

With ε-arrows: start from ε-closure(q0) (q0 + everything reachable by ε only), and after each move take the ε-closure again.

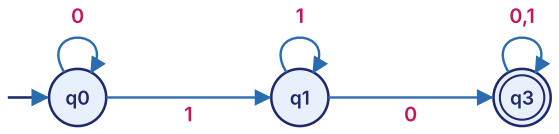
Example 1 (class photo, slides 75–77) — F = {q2}



NFA	0	1
→q0	q0	q1
q1	{q1,q2}	q1
*q2	q2	{q1,q2}

```
δ'([q0],0) = [q0]
δ'([q0],1) = [q1]
δ'([q1],0) = [q1,q2]
← new
δ'([q1],1) = [q1]
δ'([q1,q2],0) =
{q1,q2}∪{q2}
                =
[q1,q2]
δ'([q1,q2],1) = {q1}
∪{q1,q2}
                =
[q1,q2]
```

DFA	0	1
→q0	q0	q1
q1	{q1,q2} = q3	q1
*q3 = {q1,q2}	q3	q3

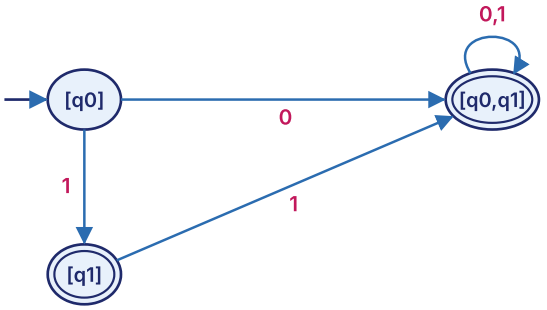


q3 = [q1,q2]

Example 2 (slide 78) — F = {q1}

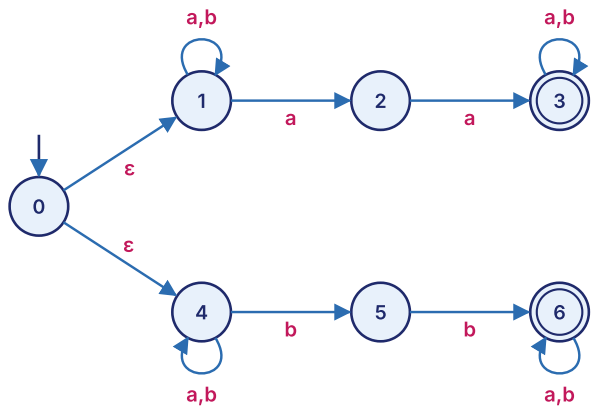
	0	1
→q0	{q0,q1}	{q1}
*q1	ϕ	{q0,q1}

```
δ'([q0],0) = [q0,q1]
← new
δ'([q0],1) = [q1]
δ'([q1],0) = ϕ
δ'([q1],1) = [q0,q1]
δ'([q0,q1],0) =
[q0,q1]
δ'([q0,q1],1) =
[q0,q1]
```



[q1] and [q0,q1] are final (they contain q1). ϕ is the dead state (often not drawn).

Application (slide 79) — ε-NFA → DFA — solved

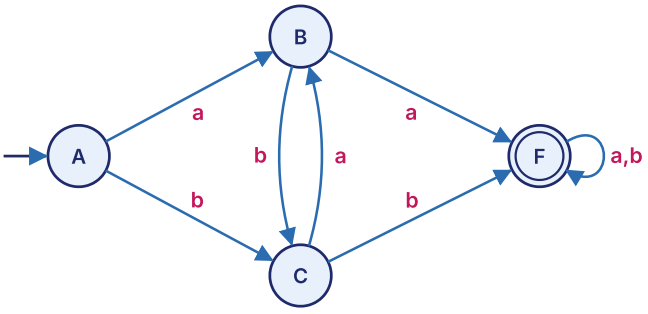


DFA state	a	b
→A = {0,1,4}	{1,2,4} = B	{1,4,5} = C
B = {1,2,4}	{1,2,3,4} = D	C
C = {1,4,5}	B	{1,4,5,6} = E
*D = {1,2,3,4}	D	{1,3,4,5} = F
*E = {1,4,5,6}	{1,2,4,6} = G	E
*F = {1,3,4,5}	D	{1,3,4,5,6} = H
*G = {1,2,4,6}	{1,2,3,4,6} = I	E
*H = {1,3,4,5,6}	I	H
*I = {1,2,3,4,6}	I	H

Language = words over {a,b} that contain **aa** or **bb**.

ε-closure(0) = {0,1,4} = **A** (start)

```
Minimise:
Step0: (D,E,F,G,H,I) (A,B,C)
Step1: (D,…,I) (A) (B) (C)
  B –a→ final, C –b→ final, A never
  ⇒ A, B, C all different
  D…I only go to D…I ⇒ fusion
  ⇒ one accepting state "F"
```



Example — moves on a 3×3 chessboard (slides 80–86)

States = squares 1…9; inputs **w** = move to an adjacent white square, **b** = adjacent black square. Start 1, final = opposite corner 9.

NFA	w	b		w	b
→1	2,4	5	6	2,8	3,5,9
2	4,6	1,3,5	7	4,8	5
3	2,6	5	8	4,6	5,7,9
4	2,8	1,5,7	*9	6,8	5
5	2,4,6,8	1,3,7,9			

DFA	w	b
→q0 = {1}	q1	q2
q1 = {2,4}	q3	q4
q2 = {5}	q3	q5
q3 = {2,4,6,8}	q3	q6
q4 = {1,3,5,7}	q3	q6
*q5 = {1,3,7,9}	q3	q2
*q6 = {1,3,5,7,9}	q3	q6

7 states q0…q6 like on her slide; finals = the sets containing 9.

Exam-day checklist

Which method for which question?

Question says…	Do…
"Give the RegEx of…"	Pattern table (Ch1 §4)
"Give the elements / words of…"	List the 4–5 shortest words + "…"
"Is this equality true?"	Counter-example word
"Build the automaton of this RegEx"	Glushkov (number → successors → draw)
"Minimise"	Remove unreachable → Step0 (F)(non-F) → split → fusion
"Is it deterministic?"	Check complete + unambiguous (+ no ε)
"Convert NFA to DFA"	Subset construction, final = contains NFA final
"Doesn't contain / complement"	Complete DFA + flip states
"Is word w accepted?"	Follow the path; must END in a final state

⚠ Mistakes that cost points

- Forgetting to mark the **initial arrow** and the **double circles**.
- Glushkov: forgetting to add **0 to F when ε ∈ L**; forgetting that x* makes the next letter a possible start/end letter.
- Minimisation: starting with anything else than **(finals)(non-finals)**; not deleting unreachable states first.
- NFA→DFA: forgetting a "new state" in the list, or forgetting that a set is final if it contains **at least one** NFA final state.
- A DFA must have **exactly one** arrow per symbol in every state → add the dead state.
- Complementing an NFA by flipping states — **wrong**, determinise first.

Tools to check yourself: **regexper.com** (RegEx → diagram) · **automatonsimulator.com**

Still to come (course outline)

The outline announces 6 chapters; the slides in this folder cover the first part. Still coming: **Turing machine** (application of automata) and **applications of automata in AI**. Add their slides to `automata/` and this sheet can be extended.

Made from the course slides (chapter1_regEx, chapter2_automata, chapter3_DFA) and the board photos of 04/11. Print: Ctrl+P → A4.